

# Docker Compose 深入速查表

从基础到生产，覆盖面试高频点和实际排障场景

## 一、核心概念与架构

### Compose 是什么

- 单机容器编排工具：一个 docker-compose.yml 定义多容器应用
- V1 vs V2: docker-compose (Python, 独立安装) → docker compose (Go, Docker 内置插件)
- 本质：读取 yaml → 解析服务依赖 → 调用 Docker API 创建容器/网络/卷

### 核心对象

| 对象       | 作用                  | 面试点                          |
|----------|---------------------|------------------------------|
| services | 定义容器（镜像、命令、端口、环境变量） | 一个 service 可运行多个实例（scale）    |
| networks | 容器间通信隔离             | 自定义网络支持 DNS 解析服务名            |
| volumes  | 数据持久化               | named volume 由 Docker 管理，可备份 |
| configs  | 配置文件挂载（Swarm 用得多）   | 敏感配置不用环境变量，用 configs         |
| secrets  | 敏感数据管理（Swarm/K8s）   | 密码、密钥独立管理                    |

## 二、docker-compose.yml 完整语法

### 基础结构模板

```
version: '3.8' # 版本声明，新版可省略

services: # 服务定义（核心）
  web: # 服务名（容器内 DNS 解析用）
    image: nginx:alpine # 镜像来源
    build: # 或从 Dockerfile 构建
      context: ./web # 构建上下文路径
      dockerfile: Dockerfile.prod # 指定 Dockerfile
    args: # 构建参数
      - VERSION=1.0
    container_name: web-01 # 自定义容器名（默认 项目名 _ 服务名 _ 序号）

# 命令覆盖
command: nginx -g 'daemon off;' # 覆盖 Dockerfile CMD
entrypoint: /entrypoint.sh # 覆盖 ENTRYPOINT

# 端口映射
ports:
  - "80:80" # 宿主机: 容器 (IPv4)
  - "127.0.0.1:8080:80" # 绑定特定 IP
  - "0.0.0.0:443:443/tcp" # 指定协议
```

```

# 环境变量
environment:                                # 方式 1: 直接写
  - NODE_ENV=production
  - DEBUG=true
env_file:                                    # 方式 2: 从文件加载
  - ../.env
  - ../.env.local

# 卷挂载
volumes:
  - ../html:/usr/share/nginx/html:ro        # bind mount + 只读
  - web-data:/data                          # named volume
  - /tmp/cache:/tmp/cache:rw               # 指定读写权限

# 网络
networks:
  - frontend                                # 自定义网络
  - backend                                # 可加入多个网络

# 依赖关系
depends_on:                                  # 控制启动顺序（不保证服务就绪）
  - db
  - redis

# 资源限制
deploy:                                     # 仅 Swarm 生效，单机用 docker run 参数
  resources:
    limits:
      cpus: '0.5'
      memory: 512M
    reservations:
      cpus: '0.25'
      memory: 256M

# 重启策略
restart: unless-stopped                    # no/on-failure/always/unless-stopped

# 健康检查
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:80/health"]
  interval: 30s                            # 检查间隔
  timeout: 10s                              # 超时时间
  retries: 3                                # 失败重试次数
  start_period: 40s                         # 启动宽限期（容器启动后多久开始检查）

# 日志配置
logging:
  driver: "json-file"
  options:
    max-size: "10m"                         # 单个日志文件大小
    max-file: "3"                           # 保留几个日志文件

```

```

# 其他
working_dir: /app          # 工作目录
user: "1000:1000"          # 运行用户
privileged: false          # 特权模式 (生产禁用)
stdin_open: true          # docker run -i
tty: true                  # docker run -t

db:
  image: mysql:8.0
  environment:
    MYSQL_ROOT_PASSWORD: ${DB_PASSWORD} # 引用 .env 变量
    MYSQL_DATABASE: myapp
  volumes:
    - db-data:/var/lib/mysql
  networks:
    - backend
# 生产级: 等 db 健康后 web 才启动
healthcheck:
  test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "root", "-p${DB_PASSWORD}"]
  interval: 5s
  timeout: 3s
  retries: 10

redis:
  image: redis:alpine
  networks:
    - backend
  volumes:
    - redis-data:/data
  command: redis-server --appendonly yes # 启用 AOF 持久化

# 网络声明
networks:
  frontend:
    driver: bridge
    ipam:
      # IP 地址管理
      config:
        - subnet: 172.20.0.0/16
          gateway: 172.20.0.1
  backend:
    driver: bridge
    internal: true # 内部网络, 不连接外网 (安全)

# 卷声明
volumes:
  web-data:
    driver: local
  db-data:
    driver: local
  redis-data:

```

driver: local

### 三、常用命令速查

#### 生命周期

| 命令                                 | 作用               | 面试/生产场景          |
|------------------------------------|------------------|------------------|
| docker compose up -d               | 后台启动所有服务         | 标准启动方式           |
| docker compose up -d --build       | 强制重新构建镜像后启动      | 代码更新后部署          |
| docker compose up -d --no-deps web | 启动 web 但不启动依赖    | 独立调试某个服务         |
| docker compose down                | 停止并删除容器 + 网络     | 清理环境             |
| docker compose down -v             | 停止并删除容器 + 网络 + 卷 | <b>危险</b> : 数据会丢 |
| docker compose down --rmi all      | 连镜像一起删           | 彻底清理             |
| docker compose stop                | 停止服务（保留容器）       | 临时暂停             |
| docker compose start               | 启动已停止的服务         | 恢复运行             |
| docker compose restart             | 重启服务             | 配置重载             |

#### 查看与调试

| 命令                                      | 作用                        |
|-----------------------------------------|---------------------------|
| docker compose ps                       | 查看服务状态                    |
| docker compose logs                     | 查看所有服务日志                  |
| docker compose logs -f web              | 实时跟踪 web 服务日志             |
| docker compose logs --tail 100 db       | 看 db 最后 100 行             |
| docker compose exec web sh              | 进入 web 容器                 |
| docker compose exec db mysql -u root -p | 进入 db 执行命令                |
| docker compose top                      | 查看服务进程                    |
| docker compose config                   | <b>验证 yaml 语法</b> 并输出解析结果 |
| docker compose config -q                | 仅检查语法，不输出                 |

#### 构建与镜像

| 命令                                  | 作用          |
|-------------------------------------|-------------|
| docker compose build                | 构建所有服务的镜像   |
| docker compose build web            | 只构建 web 服务  |
| docker compose build --no-cache web | 不用缓存强制构建    |
| docker compose pull                 | 拉取所有服务的最新镜像 |
| docker compose push                 | 推送构建的镜像到仓库  |

扩展与更新

| 命令                                                 | 作用                 | 场景      |
|----------------------------------------------------|--------------------|---------|
| <code>docker compose up -d --scale web=3</code>    | web 服务扩 3 个实例      | 负载测试    |
| <code>docker compose up -d --force-recreate</code> | 强制重建容器             | 配置变更后生效 |
| <code>docker compose up -d --remove-orphans</code> | 删除 compose 中已移除的服务 | 清理旧服务   |

四、面试高频问题与话术

Q1: depends\_on 能保证服务就绪吗？

答：不能。depends\_on 只控制启动顺序，不检查服务是否真正可用。

生产解决方案：

```
# 方案 1: 健康检查 + condition (compose v2.1+)
services:
  web:
    depends_on:
      db:
        condition: service_healthy
  db:
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 5s
      retries: 10

# 方案 2: 应用层重试（更可靠）
# web 应用启动时循环尝试连接 db，失败就重试
```

Q2: bind mount 和 named volume 怎么选？

| 维度  | bind mount     | named volume                      |
|-----|----------------|-----------------------------------|
| 路径  | 宿主具体路径         | Docker 管理路径                       |
| 备份  | 直接备份宿主目录       | <code>docker volume backup</code> |
| 跨主机 | 依赖宿主路径         | 可配合卷驱动跨主机                         |
| 权限  | 容易有 UID/GID 问题 | Docker 自动处理                       |
| 场景  | 开发环境代码热更新      | 生产数据持久化                           |

面试话术：

“开发环境我用 bind mount 挂载代码目录，实现热更新；生产环境用 named volume，由 Docker 管理存储位置，配合备份策略，避免宿主路径依赖。”

### Q3: compose 项目名是什么? 怎么改?

```
# 默认项目名 = 所在目录名
# 比如目录叫 my-project, 项目名就是 my-project
# 容器名默认: my-project_web_1

# 方式 1: 命令行指定
docker compose -p myapp up -d

# 方式 2: 环境变量
export COMPOSE_PROJECT_NAME=myapp
docker compose up -d

# 方式 3: .env 文件
COMPOSE_PROJECT_NAME=myapp
```

### Q4: 一个 service 怎么运行多个实例?

```
# 命令行扩容
docker compose up -d --scale web=3

# 效果: web_1, web_2, web_3 三个容器
# 配合负载均衡: nginx 反向代理到三个实例
```

注意: 多个实例不能同时绑定同一个宿主机端口, 要改配置:

```
services:
  web:
    ports:
      - "80" # 不指定宿主机端口, Docker 随机分配
```

### Q5: compose 文件怎么拆管理?

```
# 基础配置 + 环境覆盖
# docker-compose.yml # 基础配置
# docker-compose.prod.yml # 生产覆盖
# docker-compose.override.yml # 开发覆盖 (自动加载)

# 生产启动: 合并多个文件, 后面的覆盖前面的
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d
```

## 五、生产排障场景

### 场景 1: 服务起不来, 看日志

```
# 1. 看所有服务状态
docker compose ps

# 如果某个服务 Restarting 或 Exit

# 2. 看该服务日志
```

```
docker compose logs -f < 服务名 >

# 3. 看最后几行
docker compose logs --tail 50 < 服务名 >

# 4. 进入容器手动执行命令排查
docker compose exec < 服务名 > sh
```

## 场景 2：配置改了没生效

```
# 修改 docker-compose.yml 后，必须重建容器才生效
docker compose up -d --force-recreate

# 或者只重建某个服务
docker compose up -d --force-recreate web
```

## 场景 3：卷数据想清理/迁移

```
# 查看卷
docker compose ls

# 备份 named volume
docker run --rm -v myproject_db-data:/source -v $(pwd):/backup alpine tar czf /backup/
↳ db-backup.tar.gz -C /source .

# 恢复
docker run --rm -v myproject_db-data:/target -v $(pwd):/backup alpine tar xzf /backup/
↳ db-backup.tar.gz -C /target

# 注意：卷名带项目名前缀！
```

## 场景 4：网络不通，容器间 ping 不通

```
# 1. 确认在同一个网络
docker compose ps
# 看 Networks 列

# 2. 进容器测试 DNS
docker compose exec web sh
ping db # 应该用服务名能 ping 通
nslookup db # 看 DNS 解析

# 3. 如果不行，检查网络配置
docker network inspect < 项目名 >_backend

# 4. 检查防火墙/iptables
iptables -L -n | grep < 项目名 >
```

六、.env 文件最佳实践

```
# .env 文件 (compose 自动加载, 不用手动 source)
COMPOSE_PROJECT_NAME=myapp
DB_PASSWORD=secret123
REDIS_PASSWORD=secret456
NGINX_PORT=80

# 在 docker-compose.yml 中引用
services:
  db:
    environment:
      MYSQL_ROOT_PASSWORD: ${DB_PASSWORD}
    ports:
      - "${NGINX_PORT}:80"
```

安全注意:

- .env 文件不要提交到 Git, 加 .gitignore
- 生产密码用 CI/CD 环境变量注入, 不用 .env
- 敏感数据用 Docker Secrets (Swarm) 或 K8s Secrets

七、常见错误与解决

| 错误                                                     | 原因                | 解决                                         |
|--------------------------------------------------------|-------------------|--------------------------------------------|
| service "web" depends on undefined service "db"        | 服务名拼写错误           | 检查 depends_on 和 services 下的名字是否一致          |
| port is already allocated                              | 宿主机端口冲突           | 改端口映射或杀掉占用进程                               |
| Cannot create container for service web: No such image | 镜像没构建/拉取          | docker compose build 或 docker compose pull |
| yaml: line 10: did not find expected key               | yaml 缩进错误         | 检查空格, 不能用 Tab                              |
| bind mount source path does not exist                  | bind mount 源路径不存在 | 创建目录或改用 named volume                       |
| Container "xxx" is unhealthy                           | 健康检查失败            | 检查 healthcheck 命令是否正确                      |

八、面试速记口诀

```
一文件: docker-compose.yml 定义一切
二对象: services (容器) + networks (网络) + volumes (卷)
三命令: up (启动) / down (清理) / logs (看日志)
四注意: depends_on 不保证就绪 / 改配置要重建 / 卷备份要注意 / 项目名默认目录名
```

打印出来, 结合之前 Docker 基础速查表一起用, 面试前过一遍。